

Optimization of Duplication-Based Schedules on Network-on-Chip Based Multi-Processor System-on-Chips

Qi Tang, Shang-Feng Wu, Jun-Wu Shi, and Ji-Bo Wei, *Member, IEEE*

Abstract—Many applications such as streaming applications are both computation and communication intensive. The Multi-Processor System-on-Chip (MPSoC) based on Network-on-Chip (NoC) outperforms the multiprocessors with bus-based networking architecture in communication bandwidth and scalability, making it a better choice for implementing systems running these applications. It's important to schedule both the computation and communication onto processors and the networking architecture so as to satisfy the stringent timing requirements. To reduce or avoid inter-processor communication, task duplication has been employed in scheduling. Most of the available techniques for the duplication-based scheduling problem use heuristics to solve the problem, and seldom has any work studied further improving the schedule performance, despite the fact that the heuristic cannot provide quality guarantee. To fill in this gap, this paper introduces a duplication and mapping constrained task-communication co-scheduling problem that assumes the duplication strategy and task-to-processor mapping are known a priori, and proposes two Integer Linear Programming (ILP) formulations, i.e., CF-ILP and CA-ILP, to solve two editions of this problem, i.e., the contention-free problem and the contention-aware problem. The proposed ILP formulations optimize the ordering and timing of the communication and computation, thus improving the performance. Both synthesized and real applications are tested on a set of platforms to evaluate the performance of the proposed methods. The experimental results demonstrate the effectiveness of the proposed methods.

Index Terms—Network-on-chip, scheduling, integer linear programming, task duplication, communication contention

1 INTRODUCTION

MODERN applications, such as video/audio en/decoding and software defined radio, are everywhere in our daily lives. To satisfy the quality requirement of the consumer, these applications are ever growing in computation complexity and communication complexity, imposing great challenges on the hardware design, especially when it comes to consumer electronics that are usually battery powered [1]. Therefore, it's necessary to make a trade-off between computation capacity and power consumption in platform selection. Besides, these applications generally have stringent timing requirements, e.g., in terms of the latency between input and output. To meet the above requirements, Multi-Processor System-on-Chips (MPSoCs) are widely used in practice. However, traditional MPSoCs use bus-based networking architecture, which suffers the problem of high latency and low scalability. Network-on-Chips (NoCs) [2], [3], a big step towards this problem, is a novel communication architecture that provides high level of parallel communication, can better satisfy the communication requirement. Besides, the NoC is good at scalability, the communication

efficiency does not decrease sharply with the processor number, thus it is widely used in the design of MPSoC. NoC-based MPSoC not only provides parallel computation ability, but also parallel communication ability, which makes it a necessity to properly schedule both the computation and communication onto the processors and the NoC. Since communication latency has significant impact on the system performance, task duplication [4], [5], [6] has been utilized to reduce or avoid communication latency in the schedule. For practical systems, communication contention has significant influence on the schedule performance [7], making task duplication more beneficial [6].

In this paper, we study the duplication and mapping constrained task and communication co-scheduling problem on NoC-based MPSoCs with the objective of minimizing the schedule length. Since we focus on improving the performance of schedules produced by available duplication-based techniques, we assume the task duplication strategy and task-to-processor mapping are known a priori. Based on these assumptions, we propose two Integer Linear Programming (ILP) formulations, i.e., CF-ILP and CA-ILP, to solve two editions of the problem, i.e., the contention-free problem and the contention-aware problem. CF-ILP focuses on the contention-free problem and CA-ILP focuses on the contention-aware problem. These ILP formulations optimize the ordering and timing of the computation and communication given the duplication strategy and mapping. The proposed ILP formulations can be used to optimize the duplication-based schedules produced by available techniques, and they also lay the foundation for searching

- The authors are with the Department of Electronic Science and Engineering, National University of Defense Technology, Changsha, Hunan 410073, China. E-mail: q.tang.andyy@qq.com, sfwu1104@126.com, shijunwu77@163.com, wjbhw@nudt.edu.cn.

Manuscript received 19 Jan. 2016; revised 1 Aug. 2016; accepted 7 Aug. 2016.
Date of publication 10 Aug. 2016; date of current version 15 Feb. 2017.

Recommended for acceptance by M. Smith.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TPDS.2016.2599166

heuristics for the duplication-based scheduling problem. The proposed ILP formulations have tackled the resource contention for processors and communication links. What's more, they also have addressed the problem of selecting the optimal depended task instance when the depended task is duplicated and distributed to multiple processors.

The remainder of the paper is organized as follows. In Section 2, we discuss the related work. The application model and platform model are described in Sections 3 and 4. Section 5 introduces the background of scheduling. In Section 6 we formalize the problem to be solved. The ILP formulations for the duplication and mapping constrained problem are introduced in Sections 7 and 8. In Section 9 the experimental results are presented. We conclude the paper in Section 10.

2 RELATED WORK

Traditional bus-based interconnecting architecture suffers problems of low communication efficiency, long latency and insufficient scalability. To overcome these flaws of the bus-based communication architecture, Benini [8] proposed the concept of NoC. Over the last two decades, NoC has obtained a great deal of research attention, including the design of NoC [2], [3] and NoC-based MPSoC [9], and scheduling on NoC-based MPSoCs [10], [11], [12]. This paper focuses on the last research area.

Topcuoglu [13] proposed the Heterogeneous Earliest-Finish-Time (HEFT) algorithm for scheduling applications modeled by Directed Acyclic Graphs (DAGs) onto heterogeneous multiprocessor systems. HEFT is a list-based static scheduling algorithm, which comprises two steps, i.e., priority assignment and scheduling. The HEFT uses bottom level as the task priority, and schedules tasks according to the non-increasing order of their priorities. When trying to schedule a task, it is tentatively scheduled to all processors, and the task finish time on each processor is recorded. Finally, the processor that makes the task finish with the earliest time is selected and the task is scheduled to this processor. However, HEFT does not take into account communication contention, making it too ideal. As demonstrated in [7], communication contention significantly impacts the schedule performance, making it important to consider communication contention while scheduling the application. The flaw of HEFT is solved in the Contention-Aware Scheduling (CAS) algorithm [14], which takes into account communication latency and contention, therefore, CAS is more suitable for practical system design. However, these works have not considered using task duplication to ease the communication overhead.

Many available works have investigated the duplication-based scheduling problem [6], [15], showing that the task duplication strategy can improve the schedule performance since communication overhead is released using task duplication. Doruk [15] proposed a duplication based bottom-up scheduling algorithm (DBUS). DBUS is a list-based heuristic that uses critical path and static top level to assign priorities to the tasks. Tasks are scheduled in a bottom-up fashion. When scheduling a task, all processors where the children of the current task are mapped to should be covered. The task is duplicated and inserted to processors to cover all these processors where its children are deployed, so that its start time is minimized and the child can execute as it was

scheduled. However, communication contention is not considered in DBUS. For the duplication-based contention-aware scheduling problem, Sinnen [6] proposed the Contention-Aware Duplication-based algorithm (CA-D), which is the first try to combine task duplication and communication contention. Similar with DBUS, CA-D is also a list-based heuristic, using bottom level as the task priority. Different from DBUS that duplicates the current task to minimize its start time while enabling its children to fire as scheduled, CA-D schedules the task while duplicating critical parents to the same processor. When scheduling a task, the critical parent tasks are recursively duplicated to the current processor if it is beneficial in reducing the task finish time, i.e., the critical parent of the current task's critical parent is also duplicated. However, both DBUS and CA-D are heuristics that do not consider further optimization. To the contrary, this paper studies how to find the optimal solution given the duplication strategy and task-to-processor mapping so as to further optimize the schedule.

This paper investigates how to use ILP to solve the duplication and mapping constrained task-communication co-scheduling problem, with the objective of minimizing the schedule length. ILP is studied a lot for solving the scheduling problem. Since the scheduling problem is a combinatorial optimization problem, therefore, using ILP to formulate the problem is a straightforward method. Most available works related with ILP apply to nonduplication-based scheduling [12], [16]. Tosun [12] proposed an ILP formulation to minimize the energy consumption while mapping applications on NoC-based systems. The proposed ILP formulation optimizes the mapping of tasks on nodes of the NoC, so that the total communication cost in terms of energy consumption is minimized. However, the ordering and timing of the task are not solved, and the schedule length is not optimized. Venugopalan [16] introduced a mixed ILP formulation for scheduling with communication latency on the parallel system. Application-specific knowledge is utilized to eliminate the need to linearize the bi-linear equations owing to the existence of communication delays, thus reducing the number of variables and constraints and speeding up the runtime of the solver. However, communication contention and task duplication are not considered in those ILP formulations proposed in [12], [16]. Different from the techniques in available literature, the communication latency, contention and task duplication are all or partly modeled in the proposed ILP formulations in this paper. Besides, specific issues caused by task duplication are also taken into account.

3 APPLICATION MODEL

This paper targets DAGs that are widely used in application modeling. The formal definition is as follows.

Definition 1 (DAG). A DAG is a directed graph and is denoted by $G = (\mathbf{V}, \mathbf{E})$, where $\mathbf{V}$ is a finite set of nodes representing tasks of the application, and $\mathbf{E}$ is a finite set of directed edges denoting inter-task communications. Each node $v \in \mathbf{V}$ is associated with a cost $c(v)$ that represents the number of clock cycles needed to complete an execution of the task. Each edge $e \in \mathbf{E}$ is defined as a tuple (src, dst) , where src is the source task, dst is the destination task. Each edge $e \in \mathbf{E}$

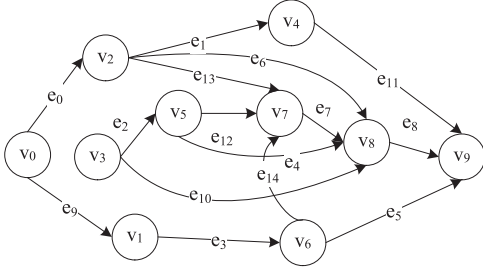


Fig. 1. The example DAG.

is associated with a cost $w(e)$ that denotes the number of data to be transferred from the source task to the destination task. For an edge e , we use notions $src(e)$, $dst(e)$ to denote its elements. We also refer to edge e as an output edge of task $src(e)$ and an input edge of task $dst(e)$.

Fig. 1 shows an example DAG of ten tasks, with the task execution time being 7, 8, 2, 4, 5, 7, 7, 4, 1, 2 and the data size of each edge being 1, 10, 9, 4, 5, 8, 5, 1, 10, 8, 2, 10, 8, 8, 1. In the remainder of this paper, we use this DAG as the example.

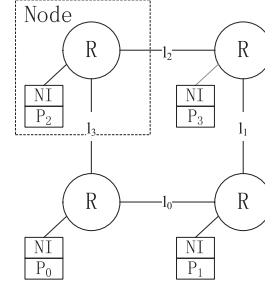
4 PLATFORM MODEL

The target platform architecture is a kind of general MPSoC interconnected by the 2D mesh NoC, which is also used in the literature [9], [11]. The platform model is defined as follows.

Definition 2 (2D NoC based MPSoC). A 2D NoC based MPSoC is composed of a set of nodes interconnected by a 2D mesh. It is denoted by $NM = (N, L)$, where N represents the set of nodes in the network. For a $x \times y$ mesh, $N = \{n_{i,j} | i \in \{1, 2, \dots, x\}, j \in \{1, 2, \dots, y\}\}$. Each node is composed of a router, a processor and a network interface. We use $P = \{p_{i,j} | i \in \{1, 2, \dots, x\}, j \in \{1, 2, \dots, y\}\}$ to represent the set of processors on NM , where $p_{i,j}$ denotes the processor on node $n_{i,j}$. $L = \{l_{i,j,r,t} | i, r \in \{1, 2, \dots, x\}, j, t \in \{1, 2, \dots, y\}\}$ represents the set of routes between different nodes, e.g., $l_{i,j,r,t}$ denotes the route from node $n_{i,j}$ to $n_{r,t}$.

Data communication in the NoC has two styles, i.e., connection-free and connection-oriented [2]. Connectionless packet switching NoCs typically do not offer latency and bandwidth guarantees, providing only the best-effort service. While the connection-oriented, circuit-switching NoCs, e.g., the dAElite [2], establish the route before data transportation, thus providing guaranteed bandwidth and latency. In this paper, we consider connection-oriented, circuit-switching NoCs, so that the timing behaviours can be accurately analyzed.

To avoid communication contention, communications using routes that partly or totally overlap should mutually exclude in transportation time, i.e., only one communication transaction is allowed to transfer on conflict routes at each time. Since the NoC in this paper uses connection-oriented, circuit-switching mechanism, communications are guaranteed to be contention-free. However, while constructing the schedule, it's necessary to consider potential contention to optimize the schedule. For the sake of simplicity in elaboration, we suppose that the NoC uses XY routing [17] and each communication transaction is transferred first in the X-axis and then in the Y-axis.



taken into account. So, the task-to-processor mapping $map(v)$, task start time $s(v)$ and edge start time $s(e)$ should be determined. The objective of scheduling is to minimize the schedule length that equals the maximum task finish time, i.e., $sl = \max_{v \in V} f(v)$. Several constraints should be satisfied to make the schedule feasible. The first constraint is the computation resource constraint, as captured by Equation (4). This constraint makes tasks mapped to the same processor execute in serial to avoid processor contention

$$\forall v_i \neq v_j, map(v_i) = map(v_j) \Rightarrow f(v_i) \leq s(v_j) \text{ or } f(v_j) \leq s(v_i). \quad (4)$$

The data precedence indicated by the communication edge in the DAG should be obeyed in the schedule, as captured by Equations (5) and (6). Equation (5) forces the communication of the edge start after the source task finishes, and Equation (6) makes the destination task start after the communication finishes

$$f(src(e)) \leq s(e), \forall e \in E \quad (5)$$

$$f(e) \leq s(dst(e)), \forall e \in E. \quad (6)$$

Since the communication transaction is assumed to be non-interrupted, the finish time of the communication edge can be expressed as

$$f(e) = s(e) + delay(l_e, w(e)), \forall e \in E, \quad (7)$$

where l_e represents the route between processors where the source task and the destination task of edge e are mapped to. The delay of the communication edge can be calculated by Equation (3).

Communication contention is a critical part for application scheduling. Without considering communication contention, the obtained schedule is unrealistic and inaccurate [18]. To make the schedule more accurate, possible contentions in the NoC should be explicitly taken into account while scheduling the communication edge. In the NoC, the one-hop link used by each edge is treated as an independent resource, e.g., l_0, l_1, l_2, l_3 in Fig. 2, so the communication edges share the same link should be transported sequentially, as captured in the following equation:

$$\forall e_i \neq e_j, e_i, e_j \text{ share resource} \Rightarrow f(e_i) \leq s(e_j) \text{ or } f(e_j) \leq s(e_i). \quad (8)$$

For example, in Fig. 2, the route from p_0 to p_1 is (l_0) , and the route from p_0 to p_2 is (l_0, l_1) . These two routes share the same link l_0 , so the communication edges deployed to these two routes should be transported exclusively.

Task duplication is a well-known technique used to reduce communication overhead by scheduling certain tasks on more than one processor. The communication that originates from a duplicated task and terminates at the task on the same processor is made local, thus avoiding costly inter-processor communication and reducing the schedule length by trading off computation resources. Task duplication reformulates data precedences of the DAG. In the duplication-based schedule, given a communication $e(v_a, v_b) \in E$, each duplication of v_b can start when at least one duplication of v_a has sent the output data to it.

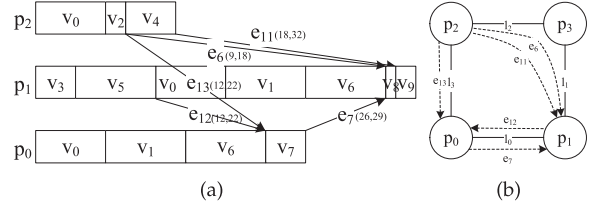


Fig. 3. The duplication-based schedule of the example DAG on the 2×2 NoC (a) and the inter-processor communication (b).

Fig. 3a depicts the duplication-based contention-aware schedule of the example DAG on the 2×2 NoC. In the schedule, v_0, v_1 and v_6 are duplicated to multiple processors, i.e., $v_0 \rightarrow (p_0, p_1, p_2)$, $v_1, v_6 \rightarrow (p_0, p_1)$. By duplicating task v_0 to these processors, communications originate from v_0 to v_1 and v_2 are made local and delay-free. The arrows in Fig. 3a depict the inter-processor communications, with the name of the corresponding edge and communication time beside the arrow, and the route used by each inter-processor communication edge is depicted in Fig. 3b using dotted lines. The source tasks of edge e_6 and e_{11} are v_2 and v_4 , both of which are mapped to processor p_2 , and their destination tasks are v_8 and v_9 that are mapped to p_1 . So both e_6 and e_{11} use the route (l_2, l_1) according to the XY routing strategy, and the corresponding communication transactions should not incur contention. In this example, the data size of edge e_6 and e_{11} are 5 and 10. Since the route (l_2, l_1) is with two hops, the delay of e_6 and e_{11} are 9 and 14 respectively according to Equation (3). So the transportation of e_6 can start immediately after v_2 finishes, i.e., at time 9, and finishes at time 18. However, the transportation of e_{11} is delayed to time 18 when the transportation of e_6 finishes and the occupation of route (l_2, l_1) is freed.

6 PROBLEM STATEMENT

Scheduling is categorized as static Scheduling and dynamic scheduling. For static scheduling [19], the firing times of all tasks and communication edges are determined before starting the application; while for dynamic scheduling, the firing times are determined in the run time. This paper assumes that the task execution time, communication time and data dependencies are known a priori, with these compile-time information, the static scheduling may outperform the dynamic scheduling. So this paper studies the static scheduling. Specifically, we focus on the duplication-based scheduling.

Most of the available techniques for duplication-based scheduling are heuristic-based, without guaranteeing the optimality of the solution. This paper studies how to optimize the duplication-based schedules on NoC-based multiprocessors. Since the communication is a critical part of the schedule in practical systems, communication latency and/or communication contention on the NoC are taken into account. For the communication routing on the NoC, the static routing strategy, i.e., XY routing [17], is used in the paper. The same as dAElite [2], we use connection-oriented, circuit switching based NoC to guarantee conflict-free communication and provide predictable services. Therefore, communications on conflict routes should be mutually exclusive and non-preemptive.

Since this work is based on the available duplication strategy and task-to-processor mapping, we define the task duplication strategy and duplication-based mapping as follows.

Definition 3 (Task duplication strategy). For a given DAG $G = (\mathbf{V}, \mathbf{E})$, the task duplication strategy is the copy number of each task in a schedule. The task duplication strategy is represented by a vector $\mathbf{R}$. $\mathbf{R}$ has $|\mathbf{V}|$ elements, and each element represents the copy number of the corresponding task in the schedule. We use $r(v)$ to denote the copy number of task v .

Definition 4 (Duplication-based mapping). For a given DAG $G = (\mathbf{V}, \mathbf{E})$ and a set of processors $\mathbf{P}$, the mapping of $\mathbf{V}$ onto $\mathbf{P}$ with task duplication strategy $\mathbf{R}$ is a function map: $\mathbf{V}_d \rightarrow \mathbf{P}$, where $\mathbf{V}_d = \{v_i^j | v_i \in \mathbf{V}, j \in \{1, 2, \dots, r(v_i)\}\}$, v_i^j denotes the j th duplication or instance of task $v_i \in \mathbf{V}$ in the mapping, and $\text{map}(v_i^j)$ represents the processor where task instance v_i^j is allocated to.

A good duplication-based mapping should not map more than one instance of a task to the same processor, i.e., $\forall v_i \in \mathbf{V}, j \neq k, j, k \in \{1, 2, \dots, r(v_i)\}, \text{map}(v_i^j) \neq \text{map}(v_i^k)$. The primal function of task duplication is reducing costly inter-processor communication, however, allocating more than one copy of a task to the same processor cannot further reduce communication overhead. For a schedule in which task a is duplicated twice (a_1, a_2) on a processor, and a_1 executes before a_2 , then any task that depends on the output data of a_2 can be modified to depend on the output data of a_1 without increasing the schedule length, since the communication transaction can start earlier by such a modification. So the execution of a_2 is redundant. Therefore, we only consider schedules that do not violate this constraint. Some available duplication-based scheduling algorithms such as DBUS [15] and CA-D [6] also obey this constraint.

Task duplication affects the formulation of the precedence constraint of the original application. Given an edge $e \in \mathbf{E}$, the destination task of this edge cannot start until the depended data is ready in the classic scheduling, i.e., $s(\text{dst}(e)) \geq f(e)$. However, in duplication-based scheduling, both the source task and destination task of the edge may be duplicated multiple times. In the duplication-based schedule, the execution of a copy of the destination task does not depend on all copies of the source task even though any instance of the source task produces the depended data needed by each instance of the destination task, i.e., not all output data of the copies of the source task should be transported to each copy of the destination task. Even though the fan-in fan-out mechanism still works for the duplication-based scheduling problem, it would congest the NoC, increase the communication delay and the schedule length. So, to make the duplication-based schedule feasible and efficient, only one instance of the source task should be chosen to send data to each instance of the destination task. For example, in Fig. 3a, even though task v_0 is also duplicated to processor p_2 for execution, the output data of v_0 on p_2 does not need to be send to v_1 on p_0 .

This paper studies two editions of the scheduling problem as illustrated above, i.e., the contention-free problem and the contention-aware problem, as the DBUS [15] and

CA-D algorithm [6] do. For the contention-aware problem, communications using conflict routes should not overlap in transfer time, i.e., the route should be contention-free; while for the contention-free problem, no such constraint is enforced.

7 CONTENTION-FREE INTEGER LINEAR PROGRAMMING FORMULATION

This section introduces the Contention-Free ILP formulation for the task duplication and mapping constrained contention-free computation and communication co-scheduling problem. It's assumed that the task duplication strategy is known a priori, and the task-to-processor mapping is already determined as in literature [20]. The proposed ILP formulation provides the mechanism to optimize the ordering and timing of tasks mapped to each processor and also the schedule of communications. The task duplication strategy and task-to-processor mapping can be obtained using the DBUS algorithm [15].

7.1 Preliminaries

Before introducing the ILP formulation, the relevant notations are presented first.

Since tasks are duplicated in the schedule, the execution time of each task instance should be determined. In the ILP formulation, we introduce symbols $s(v_i^j)$ and $f(v_i^j)$ for modeling the task execution for the j th duplication of task v_i .

- $s(v_i^j)$: the firing time of task instance $v_i^j \in \mathbf{V}_d$.
- $f(v_i^j)$: the finish time of task instance $v_i^j \in \mathbf{V}_d$. The value satisfies $f(v_i^j) = s(v_i^j) + c(v_i)$.

The data precedence implied by the communication edge of the DAG is reformulated owing to task duplication. In the duplication-based schedule, there may be multiple communication edges rather than one corresponding to an original communication edge in the DAG. The number of reformulated communication edges corresponding to an edge of the DAG is the same as the number of duplications of the destination task of the edge. The reformulated communication edge is not known exactly beforehand since where the communication originates from has not been determined if the source task of the original edge is duplicated multiple times. For example, in the duplication-based schedule, for an edge $e_{v_a v_b} = (v_a, v_b) \in \mathbf{E}$, task v_a and v_b are duplicated $r(v_a)$ and $r(v_b)$ times respectively, so, it has to be determined the output data of which instance of v_a should be send to each instance of v_b . Therefore, the communication edge $e_{v_a v_b}$ is reformulated as $r(v_b)$ communication edges in the duplication-based schedule, and we call each one of them a reformulated communication edge in this paper.

To model the reformulated communication edge, we introduce symbols $e_{v_a v_b^j}$, $s(e_{v_a v_b^j})$, $f(e_{v_a v_b^j})$ and binary auxiliary variable $dep_{v_a v_b^j}$.

- $e_{v_a v_b^j}$: the reformulated communication edge from task v_a to task instance v_b^j .
- $s(e_{v_a v_b^j})$: the start time of the communication transaction of $e_{v_a v_b^j}$.

- $f(e_{v_a v_b^j})$: the finish time of the communication transaction of $e_{v_a v_b^j}$.
- $dep_{v_a^i v_b^j}$: equals 1 if v_b^j depends on the output data of v_a^i ; otherwise it equals 0.

For the reformulated communication edge $e_{v_a v_b^j}$, the task instance of task v_a that generates the input data for task instance v_b^j is not dictated. This information should be further determined in the case when task v_a is duplicated multiple times. However, using $e_{v_a v_b^j}$ to model the uncertain communication dependency between one instance of v_a and v_b^j would not incur confusion, since we force each duplication of the destination task of an edge receives the output data of only one duplication of the source task. In the following, we would show how to solve this uncertainty using linear constraints.

When the task instance v_a^i of task v_a has been chosen to send data to v_b^j , we also use $e_{v_a^i v_b^j}$, $s(e_{v_a^i v_b^j})$ and $f(e_{v_a^i v_b^j})$ to denote the corresponding communication edge, the start time and finish time.

We define set $E_r = \{e_{v_a v_b^j} | e_{v_a v_b^j} \in E, j \in \{1, 2, \dots, r(v_b)\}\}$, in which $e_{v_a v_b^j}$ denotes an edge originates from task v_a and terminates at task v_b^j in E . E_r denotes all reformulated communication edges given the duplication strategy. For each edge $e_{v_a v_b^j} \in E_r$, to model which instance of v_a produces the output data for v_b^j , we use the binary auxiliary variable $dep_{v_a^i v_b^j}$ for each task instance v_a^i , $i \in \{1, 2, \dots, r(v_a)\}$. The variable $dep_{v_a^i v_b^j}$ indicates if the task instance v_b^j depends on the data generated by task instance v_a^i . If $dep_{v_a^i v_b^j} = 1$, it means that v_b^j depends on the output data of v_a^i ; otherwise, there is no such relation and the executions of v_a^i and v_b^j are not constrained. Since for each edge $e_{v_a v_b^j} \in E_r$, v_b^j only needs to consume data from only one task instance of v_a , the sum of $dep_{v_a^i v_b^j}$ over all $i \in \{1, 2, \dots, r(v_a)\}$ should be one, as captured by the following constraint:

$$\sum_{i \in \{1, 2, \dots, r(v_a)\}} dep_{v_a^i v_b^j} = 1, \forall e_{v_a v_b^j} \in E_r. \quad (9)$$

7.2 Objective Function

The objective is to find the schedule that minimizes the schedule length. The schedule length is defined as the maximum finish time of the tasks minus the minimum firing time. Since we assume the minimum firing time is zero, i.e., $\min_{v_i^j \in V_d} s(v_i^j) = 0$, so the schedule length can be represented as $sl = \max_{v_i^j \in V_d} f(v_i^j)$. To linearize it, introduce the following constraint:

$$f(v_i^j) \leq sl, \forall v_i^j \in V_d. \quad (10)$$

So, the optimization objective can be represented as

$$\min : sl. \quad (11)$$

7.3 Task Exclusion

On each processor, tasks mapped onto it should not overlap in execution time, so that the processor only processes one task at each time. We use $V_p = \{v | map(v) = p, v \in V_d\}$ to represent the set of task instances bound to processor p . For each $v_{p1}, v_{p2} \in V_p$, $v_{p1} \neq v_{p2}$, either inequality (12) or (13) should be satisfied

$$f(v_{p1}) \leq s(v_{p2}) \quad (12)$$

$$f(v_{p2}) \leq s(v_{p1}). \quad (13)$$

The constraint “either inequality (12) or (13) should be satisfied” belongs to *either-or* constraints, which are non-linear. To transform it to linear constraint, we introduce an auxiliary binary variable $z_{p1,p2}$ and a large positive integer M_1 . Variable $z_{p1,p2}$ equals 0 if inequality (12) holds and equals 1 if inequality (13) holds. Then the *either-or* constraint can be substituted by constraints

$$f(v_{p1}) - s(v_{p2}) \leq M_1 * z_{p1,p2} \quad (14)$$

$$f(v_{p2}) - s(v_{p1}) \leq M_1 * (1 - z_{p1,p2}), \quad (15)$$

where M_1 should be large enough such that inequations (16) and (17) always hold

$$f(v_{p1}) - s(v_{p2}) \leq M_1 \quad (16)$$

$$f(v_{p2}) - s(v_{p1}) \leq M_1. \quad (17)$$

If $z_{p1,p2} = 0$, inequality (14) makes inequality (12) hold; and (15) also holds since inequality (17) always holds, so the *either-or* constraint holds if $z_{p1,p2} = 0$. Similarly, if $z_{p1,p2} = 1$, it can be proven that the *either-or* constraint also holds.

7.4 Data Precedences

As introduced before, variable $dep_{v_a^i v_b^j}$ is used to model if v_b^j depends on v_a^i . If $dep_{v_a^i v_b^j} = 1$, then the data precedence between v_a^i and v_b^j should be satisfied. Since in the CF-ILP model we only consider communication delay, the communication contention is ignored as DBUS does, therefore, the transportation of the output data of v_a^i can start immediately after v_a^i finishes execution, so the data precedence relation between v_b^j and v_a^i can be expressed as the following *if-then* constraint:

$$\forall e_{v_a v_b^j} \in E_r, i \in \{1, 2, \dots, r(v_a)\}, dep_{v_a^i v_b^j} = 1 \Rightarrow f(v_a^i) + delay(l_{e_{v_a^i v_b^j}}, w(e_{v_a v_b^j})) \leq s(v_b^j), \quad (18)$$

where $l_{e_{v_a^i v_b^j}}$ denotes the route used by edge $e_{v_a^i v_b^j}$.

The value of $dep_{v_a^i v_b^j}$ is not known, as the source task instance that provides input data has to be chosen from a set of task instances owing to duplication. This uncertainty makes the above constraint non-linear, therefore, it is necessary to linearize the constraint. We introduce a large positive integer M_2 , and transform the constraint to the following linear constraint:

$$f(v_a^i) - s(v_b^j) + delay(l_{e_{v_a^i v_b^j}}, w(e_{v_a v_b^j})) \leq M_2 * (1 - dep_{v_a^i v_b^j}), \quad (19)$$

where M_2 should be large enough such that Equation (20) always holds

$$f(v_a^i) - s(v_b^j) + \text{delay}(l_{e_{v_a v_b}^{i,j}}, w(e_{v_a v_b})) \leq M_2. \quad (20)$$

When $\text{dep}_{v_a v_b}^{i,j}$ equals 1, Equation (19) is reduced to the right side of Equation (18), so the *if-then* constraint is satisfied; otherwise, the value of the left side of Equation (19) is not limited as guaranteed by Equation (20) that holds for any case, so the execution of the source task, destination task are not constrained if $\text{dep}_{v_a v_b}^{i,j} = 0$. Therefore, the transformation from Equation (18) to Equations (19) and (20) does not change the behavior of the *if-then* constraint.

8 CONTENTION-AWARE INTEGER LINEAR PROGRAMMING FORMULATION

In practical systems, communication contention is often unavoidable. Take the communication route in the NoC for example, multiple communication transactions share the route need to be transported in sequential to pass through the route. So, assuming multiple communication transactions can traverse a route in parallel underestimates the communication delay, since communication contention may postpone the start time of the communication transaction and result in extra delay. Besides, the extent of the underestimation grows with the volume of the communication edge that shares the same link, the delay of the link and router, and the length of the route.

Available literature uses edge scheduling, i.e., the scheduling of the communication edge onto the communication route, to make the schedule aware of communication contention. Similar with task scheduling, in which each task exclusively occupies the processor for execution, communication routes are treated as sharable resources in edge scheduling and each communication edge exclusively occupies a specific communication route for data transportation. So, the transportation of communication transactions sharing a route should be arbitrated when requiring the sharable resource at the same time so as to avoid communication contention.

This section refines the CF-ILP model for the contention-aware task duplication and mapping constrained computation and communication scheduling problem. It is assumed that the task duplication strategy and the duplication-based mapping are known a priori by using available techniques, e.g., the CA-D [6] algorithm. Based on the task duplication strategy and the valid duplication-based mapping, we model the communication contention into the ILP model, thus making the schedule contention-aware.

The major difference between the CF-ILP model and the CA-ILP model is that the latter takes into account communication contention, i.e., communications using conflict routes exclude each other in transfer time. However, communication transactions need to be transported are not clear beforehand. Being aware that tasks may be duplicated to multiple processors in the duplication-based schedule, so there is uncertainty about the reformulation communication edge as to where the communication transaction originates, i.e., we have no idea as to where the task produces the input data for the destination task is mapped to. Without this information, the start time and delay of the communication

transaction are unknown. Since it is known how many times the source task of a reformulation communication edge is duplicated and where each copy of the source task is mapped to, it's possible to model each communication transaction in the ILP model. In the following, we show how to model the data precedence related to each reformulated communication edge, and how to avoid conflict communication using linear constraints in the ILP model while the communication uncertainty described above exists.

For each edge $e_{v_a v_b}^{i,j}$ in the reformulated communication edge set $\mathbf{E}_r = \{e_{v_a v_b}^{i,j} | e_{v_a v_b} \in \mathbf{E}, j \in \{1, 2, \dots, r(v_b)\}\}$, the source task instance of the edge is not known a priori and there are $r(v_a)$ possible cases, we call each $e_{v_a v_b}^{i,j}$ the possible edge instance of edge $e_{v_a v_b}$.

Different from the CF-ILP where the start time of each reformulated communication edge can start immediately after its source task finishes execution since communication contention is not considered, in CA-ILP, the schedule of the communication edge should be modeled explicitly owing to contention.

As introduced before, $\text{dep}_{v_a v_b}^{i,j}$ is used to determine which task instance of v_a is depended by v_b^j . If $\text{dep}_{v_a v_b}^{i,j} = 1$, then the precedence relation between v_a^i and v_b^j should be satisfied, i.e., the start time of the reformulated communication edge $e_{v_a v_b}^{i,j}$ should be late than the finish time of task instance v_a^i , and the start time of task instance v_b^j should be late than the finish time of $e_{v_a v_b}^{i,j}$, as captured by the following constraints:

$$f(v_a^i) \leq s(e_{v_a v_b}^{i,j}) \quad (21)$$

$$f(e_{v_a v_b}^{i,j}) \leq s(v_b^j), \quad (22)$$

where $f(e_{v_a v_b}^{i,j}) = s(e_{v_a v_b}^{i,j}) + \text{delay}(l_{e_{v_a v_b}^{i,j}}, w(e_{v_a v_b}))$, $l_{e_{v_a v_b}^{i,j}}$ denotes the route used by edge $e_{v_a v_b}^{i,j}$.

The value of $\text{dep}_{v_a v_b}^{i,j}$ is not known, as the source task instance that provides input data has to be chosen from a set of task instances owing to duplication. This uncertainty makes the above constraint non-linear, therefore, it is necessary to linearize the constraint. We introduce a large positive integer M_3 , and transform the constraint to the following linear constraint:

$$f(v_a^i) - s(e_{v_a v_b}^{i,j}) \leq M_3 * (1 - \text{dep}_{v_a v_b}^{i,j}) \quad (23)$$

$$f(e_{v_a v_b}^{i,j}) - s(v_b^j) \leq M_3 * (1 - \text{dep}_{v_a v_b}^{i,j}), \quad (24)$$

where M_3 should be large enough such that Equations (25) and (26) always hold

$$f(v_a^i) - s(e_{v_a v_b}^{i,j}) \leq M_3 \quad (25)$$

$$f(e_{v_a v_b}^{i,j}) - s(v_b^j) \leq M_3. \quad (26)$$

When $\text{dep}_{v_a v_b}^{i,j}$ equals 1, Equations (23) and (24) is reduced to (21) and (22), so the constraint is satisfied; Otherwise, the values of the left side of Equations (21) and (22) are not limited as guaranteed by Equations (25) and (26), so the execution of

the source task, destination task and communication are not constrained if $dep_{v_a v_b}^{i,j} = 0$. So, the above transformation does not change the behavior of the *if-then* constraint.

For each possible edge instance, the route used can be determined using the XY routing strategy since the mapping information of the source task instance and destination task instance are known a priori. For each pair of possible edge instances belong to different reformulated communication edges in E_r , if their routes conflict and data precedences presented by these edges hold, then the transfer times of these two possible edge instances should be mutually exclusive. So, $\forall e_{v_a v_b}^{i,j} \neq e_{v_c v_d}^{k,l}, e_{v_a v_b}^{i,j}, e_{v_c v_d}^{k,l} \in E_r, \forall i \in \{1, 2, \dots, r(a)\}, k \in \{1, 2, \dots, r(c)\}$, if $dep_{v_a v_b}^{i,j} = 1, dep_{v_c v_d}^{k,l} = 1$ and route $l_{v_a v_b}^{i,j}$ conflicts with route $l_{v_c v_d}^{k,l}$, then the communications of $e_{v_a v_b}^{i,j}$ and $e_{v_c v_d}^{k,l}$ exclude in time.

For edge $e_{v_a v_b} \in E$, only one instance of task v_a should be chosen to be the precedence task of each instance of task v_b in the schedule, as captured by Equation (9). If all the generated copies of data produced by instances of task v_a are transferred to each instances of task v_b , the NoC would be blocked to a deeper extent, thus increasing the schedule length. We use $e_{v_a v_b}$ to represent the communication transaction from task v_a to the instance v_b^j of task v_b , and use $e_{v_a v_b}^{i,j}$ and $e_{v_c v_d}^{k,l}$ to substitute $e_{v_a v_b}^{i,j}$ and $e_{v_c v_d}^{k,l}$. Since whether routes $l_{v_a v_b}^{i,j}, l_{v_c v_d}^{k,l}$ conflict or not can be derived from the mapping strategy, the constraint can be expressed as Equation (27), which should hold for all pairs of possible edge instances using conflict routes

$$\left. \begin{array}{l} dep_{v_a v_b}^{i,j} = 1 \\ dep_{v_c v_d}^{k,l} = 1 \end{array} \right\} \Rightarrow \left\{ \begin{array}{l} f(e_{v_c v_d}^{k,l}) \leq s(e_{v_a v_b}^{i,j}) \text{ or} \\ f(e_{v_a v_b}^{i,j}) \leq s(e_{v_c v_d}^{k,l}). \end{array} \right. \quad (27)$$

The constraint represented by Equation (27) is an embedded *if-then-either-or* constraint, in which the constraint implied by the expression of *either-or* should be satisfied when the specific condition occurs. The *if-then-either-or* constraint is non-linear. To linearize it, we introduce an auxiliary binary variable $y_{v_a v_b}^{i,j} v_c v_d^{k,l}$ and a positive integer M_4 , then the constraint can be linearized as constraints

$$\begin{aligned} f(e_{v_c v_d}^{k,l}) - s(e_{v_a v_b}^{i,j}) &\leq M_4 * y_{v_a v_b}^{i,j} v_c v_d^{k,l} \\ &+ M_4 * (2 - dep_{v_a v_b}^{i,j} - dep_{v_c v_d}^{k,l}) \end{aligned} \quad (28)$$

$$\begin{aligned} f(e_{v_a v_b}^{i,j}) - s(e_{v_c v_d}^{k,l}) &\leq M_4 * (1 - y_{v_a v_b}^{i,j} v_c v_d^{k,l}) \\ &+ M_4 * (2 - dep_{v_a v_b}^{i,j} - dep_{v_c v_d}^{k,l}), \end{aligned} \quad (29)$$

where $f(e_{v_a v_b}^{i,j}), f(e_{v_c v_d}^{k,l})$ is defined as (30) and (31), and M_4 should be large enough such that Equations (32) and (33) always hold

$$f(e_{v_a v_b}^{i,j}) = s(e_{v_a v_b}^{i,j}) + delay(l_{v_a v_b}^{i,j}, w(e_{v_a v_b})) \quad (30)$$

$$f(e_{v_c v_d}^{k,l}) = s(e_{v_c v_d}^{k,l}) + delay(l_{v_c v_d}^{k,l}, w(e_{v_c v_d})) \quad (31)$$

$$f(e_{v_c v_d}^{k,l}) - s(e_{v_a v_b}^{i,j}) \leq M_4 \quad (32)$$

$$f(e_{v_a v_b}^{i,j}) - s(e_{v_c v_d}^{k,l}) \leq M_4. \quad (33)$$

The right sides of constraints (28) and (29) comprise two parts. The first part is used to make sure that communications of $e_{v_c v_d}^{k,l}$ and $e_{v_a v_b}^{i,j}$ meet the precedence relation. The second part is for modeling if values of $dep_{v_a v_b}^{i,j}$ and $dep_{v_c v_d}^{k,l}$ both equal 1 as the *if-then-either-or* constraint requires.

The auxiliary binary variable $y_{v_a v_b}^{i,j} v_c v_d^{k,l}$ is used for modeling the precedence relation of the communications of $e_{v_c v_d}^{k,l}$ and $e_{v_a v_b}^{i,j}$. When the condition $dep_{v_a v_b}^{i,j} = dep_{v_c v_d}^{k,l} = 1$ holds, $f(e_{v_a v_b}^{i,j}) \leq s(e_{v_c v_d}^{k,l})$ implies $y_{v_a v_b}^{i,j} v_c v_d^{k,l} = 1$, and $f(e_{v_c v_d}^{k,l}) \leq s(e_{v_a v_b}^{i,j})$ implies $y_{v_a v_b}^{i,j} v_c v_d^{k,l} = 0$; otherwise, the value of $y_{v_a v_b}^{i,j} v_c v_d^{k,l}$ is meaningless.

The correctness of the transformation from (27) to (28) and (29) is proven as follows. If $dep_{v_a v_b}^{i,j} = 1$ and $dep_{v_c v_d}^{k,l} = 1$, then (28) and (29) are reduced as follows:

$$f(e_{v_c v_d}^{k,l}) - s(e_{v_a v_b}^{i,j}) \leq M_4 * y_{v_a v_b}^{i,j} v_c v_d^{k,l} \quad (34)$$

$$f(e_{v_a v_b}^{i,j}) - s(e_{v_c v_d}^{k,l}) \leq M_4 * (1 - y_{v_a v_b}^{i,j} v_c v_d^{k,l}). \quad (35)$$

Since $y_{v_a v_b}^{i,j} v_c v_d^{k,l}$ is a binary variable, if $y_{v_a v_b}^{i,j} v_c v_d^{k,l} = 1$, Equation (35) makes $f(e_{v_a v_b}^{i,j}) \leq s(e_{v_c v_d}^{k,l})$ and Equation (34) always holds owing to Equation (32); otherwise, Equation (34) makes $f(e_{v_c v_d}^{k,l}) \leq s(e_{v_a v_b}^{i,j})$ and Equation (35) always holds owing to Equation (33). So, Equations (34) and (35) make the right side of constraint (27) hold.

If $dep_{v_a v_b}^{i,j} = 0$ or $dep_{v_c v_d}^{k,l} = 0$, then $2 - dep_{v_a v_b}^{i,j} - dep_{v_c v_d}^{k,l} \geq 1$. Since $y_{v_a v_b}^{i,j} v_c v_d^{k,l}$ is a binary variable, so, the right sides of both (28) and (29) are equal to or larger than M_4 . According to Equations (32) and (33), Equations (28) and (29) always hold under such a case and the communication transactions of $e_{v_a v_b}^{i,j}$ and $e_{v_c v_d}^{k,l}$ are unrestricted.

Take $e_6 = (v_2, v_8), e_{11} = (v_4, v_9)$ of the DAG in Fig. 1 and its duplication strategy and mapping in Fig. 3a for example. We denote the copy of v_2 mapped to processor p_2 as v_2^1 , and the copy of v_8 mapped to p_1 is denoted as v_8^1 . The source tasks of edge e_{11} mapped to processor p_2 is denoted as v_4^1 , and the destination task mapped to p_1 is denoted as v_9^1 . $dep_{v_2^1 v_8^1}$ is the binary variable for modeling the precedence relation between v_2^1 and v_8^1 ; and $dep_{v_4^1 v_9^1}$ is for modeling the precedence relation between v_4^1 and v_9^1 . Since $e_{v_2^1 v_8^1}$ and $e_{v_4^1 v_9^1}$ use conflict route, Equation (27) should hold. According to Equations (28), (29), (30), (31), (32), and (33), the constraints for making Equation (27) hold are as follows:

$$\begin{aligned} f(e_{v_4^1 v_9^1}) - s(e_{v_2^1 v_8^1}) &\leq M_4 * y_{v_2^1 v_8^1} v_4^1 v_9^1 \\ &+ M_4 * (2 - dep_{v_2^1 v_8^1} - dep_{v_4^1 v_9^1}) \end{aligned} \quad (36)$$

$$\begin{aligned} f(e_{v_2^1 v_8^1}) - s(e_{v_4^1 v_9^1}) &\leq M_4 * (1 - y_{v_2^1 v_8^1} v_4^1 v_9^1) \\ &+ M_4 * (2 - dep_{v_2^1 v_8^1} - dep_{v_4^1 v_9^1}) \end{aligned} \quad (37)$$

$$f(e_{v_2^1 v_8^1}) = s(e_{v_2^1 v_8^1}) + delay(l_{e_{v_2^1 v_8^1}}, w(e_{v_2^1 v_8^1})) \quad (38)$$

$$f(e_{v_4^1 v_9^1}) = s(e_{v_4^1 v_9^1}) + delay(l_{e_{v_4^1 v_9^1}}, w(e_{v_4^1 v_9^1})) \quad (39)$$

TABLE 1
Parameters of Real Applications

App Name	Task Number	Edge Number
MP3 decoder	27	38
modem	48	83
channel equalizer	14	21
WLAN 802.11 receiver	24	28
TD-SCDMA	16	20

$$f(e_{v_4v_9^1}) - s(e_{v_2v_8^1}) \leq M_4 \quad (40)$$

$$f(e_{v_2v_8^1}) - s(e_{v_4v_9^1}) \leq M_4, \quad (41)$$

where $\text{delay}(l_{e_{v_2v_8^1}}, w(e_{v_2v_8^1})) = 9$ and $\text{delay}(l_{e_{v_4v_9^1}}, w(e_{v_4v_9^1})) = 14$. According to Equation (9), $\text{dep}_{v_2v_8^1} = \text{dep}_{v_4v_9^1} = 1$ since v_2, v_4 are not duplicated multiple times. So Equations (36) and (37) are reduced to

$$f(e_{v_4v_9^1}) - s(e_{v_2v_8^1}) \leq M_4 * y_{v_2v_8^1}^{v_4v_9^1} \quad (42)$$

$$f(e_{v_2v_8^1}) - s(e_{v_4v_9^1}) \leq M_4 * (1 - y_{v_2v_8^1}^{v_4v_9^1}). \quad (43)$$

No matter $y_{v_2v_8^1}^{v_4v_9^1} = 0$ or 1 , the *either-or* constraint of the right side of Equation (27) holds based on the above two equations, so the communication of e_6, e_{11} would be exclusive in the schedule.

9 EXPERIMENTS AND RESULTS

In this section, we evaluate the proposed methods experimentally by comparing them with DBUS [15] and CA-D [6]. In the experiment, the output of DBUS and CA-D are used as the input of CF-ILP and CA-ILP respectively, since DBUS focuses on the contention-free problem and CA-D focuses on the contention-aware problem. In [15], it's shown that DBUS outperforms HEFT, HCPFD, HCNF; and in [6], CA-D is proven to be better than LS-CS, D-CS, CA-LS, making DBUS and CA-D quite good algorithms for performance comparison. We first introduce the benchmarks used in the experiments; then, the experimental results are presented and discussed.

9.1 Benchmarks

We use some real applications for performance evaluation, including the MP3 decoder [21], modem [22], channel equalizer [23], WLAN 802.11 receiver [24] and TD-SCDMA [24], with the application parameter being shown in Table 1. Since the application may have different behaviors on different platforms, so we configure the application with different execution parameters to reveal this fact. For each real application, the computation and communication cost are configured 100 times with different CCRs, including 1, 3, 5, and the cost of each task is uniformly distributed between 50 and 200.

Limited by the variability, these real applications are not enough to cover applications that may appear in the future. For this reason, we also use a large set of synthesized DAGs to evaluate the proposed methods. The open source tool SDF3 [25] is used to synthesize DAGs with various

properties. The DAG properties such as in-degree/out-degree are all randomly generated given the corresponding average value, minimum/maximum value and variation. The in-degree and out-degree are both given the average value and variation of 2, the minimum value of 0 and the maximum value of 4. The task execution time and the edge cost are randomly generated. For each task, the task execution time is uniformly distributed between 50 and 200. The cost of the edge is controlled by the CCR. In our experiments, we generate application graphs with the task number ranging from 20 to 40, with a step of 10. For each graph size, 100 graphs are generated. Since CCR has significant impact on the performance of the algorithm, we set each application with different CCRs, with the value in the set $\{1, 3, 5\}$. Therefore, 900 different applications are used in our experiments.

Each experiment is carried out on three different platforms that have various mesh sizes. In our experiments, the mesh sizes used are 3×3 , 4×4 , 5×5 . Even though the proposed technique can be applied to platforms with more processors theoretically, the size of the ILP formulation grows with the platform size, which makes it hard to solve. Therefore, we only test the proposed techniques on small and medium platforms. The experiments are all carried out on an Intel Core i5 processor (2.60 GHz and 4 GB RAM) running 64 bits Windows 7. In the experiment, LINDO API 9.0 [26] is used to solve the ILP formulation with the timeout being set as ten minutes.

9.2 Results for Synthesized Applications

In this section, we compare CF-ILP, CA-ILP with the corresponding heuristic, i.e., DBUS and CA-D, using synthesized applications. We use normalized schedule length (NSL) to evaluate the performance of different algorithms. To make the comparison fair, the schedule length of DBUS is normalized by that produced by CF-ILP, and the schedule length of CA-D is normalized by CA-ILP. Besides, the runtime of each algorithm is also evaluated to assess its practical usage. Since the ILP-based method uses the output of the heuristic as input, when computing the runtime of the ILP-based method, the runtime of the corresponding heuristic is also taken into account. The comparison between the contention-free methods and the contention-aware methods are not recorded, which complies with results in available literature, i.e., CA-D and CA-ILP are more practical than DBUS and CF-ILP since communication contention is considered, hence having longer schedule length.

To have a global view of the performance of four algorithms, we compute the average NSL of those algorithms over all applications with different sizes, platforms and CCRs. ILP-based algorithms, i.e., CF-ILP and CA-ILP, outperform the corresponding heuristics, i.e., DBUS and CA-D. Specifically, the NSL of CA-D is 8.8 percent higher than CA-ILP averagely; and the NSL of DBUS is 5.7 percent higher than CF-ILP. It demonstrates that CF-ILP and CA-ILP can improve the performance of the corresponding heuristic. As to the runtime, the ILP-based methods consume more runtime compared with the heuristics. Specifically, the average runtime of DBUS is 0.19 s, and that of CF-ILP is 221 s; the average runtime of CA-D is 2.39 s, and that of CA-ILP is 324 s. Besides, the runtime increases with the problem size, so the ILP-based method is more suitable for small or medium sized problems. It

TABLE 2
Average Normalized Schedule Length with Respect
to Application Size for Synthesized Applications

App Size	DBUS	CF-ILP	CA-D	CA-ILP
20	1.085	1.000	1.060	1.000
30	1.087	1.000	1.054	1.000
40	1.090	1.000	1.057	1.000

TABLE 3
Average Normalized Schedule Length with Respect
to Mesh Size for Synthesized Applications

Mesh Size	DBUS	CF-ILP	CA-D	CA-ILP
3 × 3	1.077	1.000	1.058	1.000
4 × 4	1.095	1.000	1.060	1.000
5 × 5	1.092	1.000	1.052	1.000

TABLE 4
Average Normalized Schedule Length with Respect
to CCR for Synthesized Applications

CCR	DBUS	CF-ILP	CA-D	CA-ILP
1	1.108	1.000	1.050	1.000
3	1.075	1.000	1.056	1.000
5	1.080	1.000	1.065	1.000

should be noted that not for all problems the ILP-based method can improve the performance. For the contention-free problem, about 60 percent of the tested problem instances can be improved using the ILP-based method. While for the contention-aware problem, less problem instances can be improved, with the ratio being about 40 percent. Since the ILP formulation is based on the task duplication strategy and mapping produced by the available method, which limits the potential improvement that can be gained using further optimization provided by the ILP formulation. Besides, the ILP formulation does not consider removing task duplications that may be redundant or adding any other task duplication to further reduce the communication overhead, thus further limiting the potential improvement. The above results show that it deserves further research to model the complete duplication-based schedule using the ILP formulation while removing the enforced constraints.

To study the impact of the application size on the algorithm performance, we compare the NSL of different algorithms with respect to the graph size, with the results being shown in Table 2. From the table we can see that the NSL of DBUS is about 9.0 percent higher than the CF-ILP, and CA-D is 6.0 percent higher than CA-ILP for different applications, showing that the ILP-based methods are useful for applications of different sizes.

The effect of the platform size on the algorithm performance is also studied. Table 3 compares the average NSL of different algorithms with respect to mesh size. From the table we can see that on different platforms, the NSL of DBUS is 7.7-9.2 percent worse than CF-ILP, and CA-D is 5.2-6.0 percent worse than CA-ILP, which demonstrates that the ILP-based methods are applicable to different platforms.

To investigate the impact of CCR, Table 4 summarizes the average NSL for applications with different CCRs over

TABLE 5
Average Normalized Schedule Length over All Platforms
and CCRs for Real Applications

App Name	DBUS	CF-ILP	CA-D	CA-ILP
MP3 decoder	1.073	1.000	1.041	1.000
modem	1.078	1.000	1.037	1.000
channel equalizer	1.098	1.000	1.028	1.000
WLAN 802.11 receiver	1.092	1.000	1.032	1.000
TD-SCDMA	1.084	1.000	1.046	1.000

TABLE 6
Average Normalized Schedule Length with Respect
to Mesh Size for Real Applications

Mesh Size	DBUS	CF-ILP	CA-D	CA-ILP
3 × 3	1.084	1.000	1.045	1.000
4 × 4	1.085	1.000	1.033	1.000
5 × 5	1.092	1.000	1.033	1.000

TABLE 7
Average Normalized Schedule Length with Respect
to CCR for Real Applications

CCR	DBUS	CF-ILP	CA-D	CA-ILP
1	1.101	1.000	1.032	1.000
3	1.082	1.000	1.033	1.000
5	1.077	1.000	1.045	1.000

all applications and platforms. As shown in the table, the performance improvement by using CA-ILP increase with the value of CCR. For example, the average NSL of CA-D is 1.050 when the CCR equals 1, however, it increases to 1.065 as the CCR increases to 5. Differently, the comparison between DBUS and CF-ILP has no such trend, with the improvement ranging between 7.5-10.8 percent.

9.3 Results for Practical Applications

In this section, we compare the algorithms for some practical applications. Table 5 compares the average NSL of different algorithms over all application configurations, platforms and CCRs for each practical application. Using the ILP-based methodology, the schedule length can be reduced for real applications as for synthesized applications.

The impact of the platform size on the algorithm performance for the real application is summarized in Table 6 that compares the average NSL of different algorithms with respect to the mesh size. From the table we can see that in our experiments, the mesh size of the platform has little effect on the performance similar with the synthesized applications, since even though the processor number is increased, the application may not take full use of them in the schedule.

To investigate the effect of CCR, Table 7 summarizes the average NSL for all real applications over all platforms with respect to CCR. As shown in the table, the performance improvements by using CF-ILP and CA-ILP do not have significant relation with CCR. For different CCRs, the NSL of CA-ILP is roughly the same. However, CF-ILP and CA-ILP still outperform DBUS and CA-D for different CCRs. The CA-D is 3.3-4.5 percent worse than

CA-ILP, and the DBUS is 10.1-7.7 percent worse than CF-ILP for different CCRs.

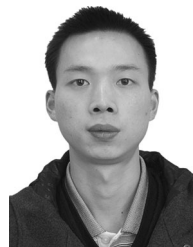
Similar with random applications, the ILP-based method takes more runtime for practical applications. According to the experiment, the average runtime of CF-ILP and CA-ILP are 169 and 255 seconds respectively, while the DBUS and CA-D only take several to tens of seconds. Besides, for some problems, the ILP formulation cannot reduce the schedule length owing to the duplication and mapping constraints made in the formulation. In the experiment, about 60 and 55 percent of the problems are improved for the contention-free problem and contention-aware problem respectively.

10 CONCLUSIONS

Two ILP formulations are proposed for the duplication and mapping constrained scheduling of applications on the NoC-based multiprocessor system with the objective of minimizing the schedule length, i.e., the CF-ILP formulation for the contention-free problem, and the CA-ILP formulation for the contention-aware problem. In the proposed ILP formulations, the reformulated data precedences incurred by task duplication, communication delay and communication contention in NoC are partly or fully modeled, thus optimizing the schedule of both the computation and communication. Sets of synthesized applications and real applications are used to evaluate the performance of the proposed methods. Experiment results demonstrate that the proposed ILP formulations can improve the performance of available duplication-based algorithms with extra runtime. Since the performance of the ILP formulation is limited by the duplication strategy and mapping that are assumed to be known a priori, in the future, we will study how to release these constraints and completely model the duplication-based scheduling problem.

REFERENCES

- [1] A. K. Singh, M. Shafique, A. Kumar, and J. Henkel, "Mapping on multi/many-core systems: Survey of current and emerging trends," in *Proc. 50th ACM/EDAC/IEEE Annu. Des. Autom. Conf.*, 2013, pp. 1–10.
- [2] R. A. Stefan, A. Molnos, and K. Goossens, "dAElite: A TDM NoC supporting QoS, multicast, and fast connection set-up," *IEEE Trans. Comput.*, vol. 63, no. 3, pp. 583–594, Mar. 2014.
- [3] K. Goossens and A. Hansson, "The aethereal network on chip after ten years: Goals, evolution, lessons, and future," in *Proc. 47th ACM/IEEE Des. Autom. Conf.*, 2010, pp. 306–311.
- [4] J. Mei, K. Li, and K. Li, "A resource-aware scheduling algorithm with reduced task duplication on heterogeneous computing systems," *J. Supercomputing*, vol. 68, no. 3, pp. 1347–1377, 2014.
- [5] X. Tang, K. Li, G. Liao, and R. Li, "List scheduling with duplication for heterogeneous computing systems," *J. Parallel Distrib. Comput.*, vol. 70, no. 4, pp. 323–329, 2010.
- [6] O. Sinnen, A. To, and M. Kaur, "Contention-aware scheduling with task duplication," *J. Parallel Distrib. Comput.*, vol. 71, no. 1, pp. 77–86, 2011.
- [7] O. Sinnen and L. A. Sousa, "Communication contention in task scheduling," *IEEE Trans. Parallel Distrib. Syst.*, vol. 16, no. 6, pp. 503–515, Jun. 2005.
- [8] L. Benini and G. D. Micheli, "Networks on chips: A new SoC paradigm," *J. Comput.*, vol. 35, pp. 70–78, 2002.
- [9] K. Goossens, et al., "Virtual execution platforms for mixed-time-criticality systems: The compSOC architecture and design flow," *ACM SIGBED Rev.*, vol. 10, no. 3, pp. 23–34, 2013.
- [10] P. K. Sahu and S. Chattopadhyay, "A survey on application mapping strategies for network-on-chip design," *J. Syst. Architecture*, vol. 59, no. 1, pp. 60–76, 2013.
- [11] J. Hu and R. Marculescu, "Energy-aware communication and task scheduling for network-on-chip architectures under real-time constraints," in *Proc. Des. Autom. Test Eur. Conf. Exhibition*, 2004, vol. 1, pp. 234–239.
- [12] S. Tosun, O. Ozturk, and M. Ozen, "An ILP formulation for application mapping onto network-on-chips," *Int. Conf. Appl. Inf. Commun. Technologies*, 2009, pp. 1–5.
- [13] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [14] A. F. Alkaya and H. R. Topcuoglu, "A task scheduling algorithm for arbitrarily-connected processors with awareness of link contention," *Cluster Comput.*, vol. 9, no. 4, pp. 417–431, 2006.
- [15] D. Bozdog, U. Catalyurek, and F. Ozguner, "A task duplication based bottom-up scheduling algorithm for heterogeneous environments," in *Proc. 20th Int. Conf. Parallel Distrib. Process.*, 2006, pp. 160–160.
- [16] S. Venugopalan and O. Sinnen, "ILP formulations for optimal task scheduling with communication delays on parallel systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 1, pp. 142–151, Jan. 2015.
- [17] S. D. Chawade, M. A. Gaikwad, and R. M. Patrikar, "Review of XY routing algorithm for network-on-chip architecture," *Int. J. Comput. Appl.*, vol. 43, pp. 20–23, 2012.
- [18] O. Sinnen, *Task Scheduling for Parallel Systems*. Hoboken, NJ, USA: Wiley, 2007.
- [19] S. Sriram and S. S. Bhattacharyya, *Embedded Multiprocessors: Scheduling and Synchronization*. Boca Raton, FL, USA: CRC Press, 2012.
- [20] L. Yang, W. Liu, W. Jiang, J. Yi, D. Liu, and Q. Zhuge, "Contention-aware task and communication co-scheduling for network-on-chip based multiprocessor system-on-chip," in *Proc. IEEE 20th Int. Conf. Embedded Real-Time Comput. Syst. Appl.*, 2014, pp. 1–8.
- [21] M. Geilen, "Synchronous dataflow scenarios," *ACM Trans. Embedded Comput. Syst.*, vol. 10, no. 2, 2010, Art. no. 16.
- [22] S. S. Bhattacharyya, P. K. Murthy, and E. A. Lee, "Synthesis of embedded software from synchronous dataflow specifications," *J. VLSI Signal Process. Syst. Signal Image Video Technol.*, vol. 21, no. 2, pp. 151–166, 1999.
- [23] S. Ritz, M. Willems, and H. Meyr, "Scheduling for optimum data memory compaction in block diagram oriented software synthesis," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 1995, vol. 4, pp. 2651–2654.
- [24] O. Moreira and H. Corporaal, *Scheduling Real-Time Streaming Applications Onto an Embedded Multiprocessor*. New York, NY, USA: Springer, 2014.
- [25] S. Stuijk, M. Geilen, and T. Basten, "SDF3: SDF for free," in *Proc. 6th Int. Conf. Appl. Concurrency Syst. Des.*, 2006, vol. 6, pp. 276–278.
- [26] LINDO API version 9.0, LINDO Systems Inc. [Online]. Available: <http://www.lindo.com/>



Qi Tang received the BSc degree in information engineering from National University of Defense Technology (NUDT), China, in 2009, and the MSc degree in information and communication engineering from NUDT, in 2012. He is currently working toward the PhD degree in the Department of Electronic Science and Engineering, NUDT. He has been awarded a scholarship from the China Scholarship Council (CSC) to conduct research in the Electronic Systems Group of the Electrical Engineering Department, Eindhoven University of Technology (TU/e) as a visiting PhD student from 2014 to 2015. His research interests include software defined radio, embedded parallel computing, and algorithm design.



Shang-Feng Wu received the BSc degree in information engineering from Beijing Institute of Technology, Beijing, China, in 2008, and the MSc degree in information and communication engineering from National University of Defense Technology (NUDT), Hunan, China, in 2011. He is currently working toward the PhD degree in the Department of Electronic Science and Engineering, NUDT. His research interests include software defined radio, computing resource management, and task mapping and scheduling algorithms design real-time systems.



Jun-Wu Shi received the BSc and MSc degrees in communication engineering from Huairou Equipment Academy and the National University of Defense Technology (NUDT), in 1998 and 2003, respectively. He is currently a lecturer in the Department of Communication Engineering, NUDT. His research interests include software defined radio and computing resource management.



Ji-Bo Wei (M'02) received the BSc and MSc degrees in electronic engineering from National University of Defense Technology (NUDT), Changsha, China, in 1989 and 1992, respectively, and the PhD degree in electronic engineering from Southeast University, Nanjing, China, in 1998. He is currently a professor in the Department of Communication Engineering, NUDT. His research interests include wireless network protocol and signal processing in communications, more specially in the areas of MIMO, multicarrier transmission, cooperative communication, and cognitive network. He is an editor of the *Journal on Communications* and the senior member of China Institute of Communications and Electronics, respectively. He is the member of the IEEE, the IEEE Communication Society, and the IEEE Vehicular Technology Society.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.**